

PAC-MAN GAME

A fully playable Pac-Man built from scratch in Java—no database, just Swing and pure OOP.

● Week 1 Complete	Java JDK 21	VS Code	Swing / JPanel	60 FPS Game Loop
-------------------	-------------	---------	----------------	------------------

WHAT WAS BUILT

Project Structure ✓ App.java — main entry point, launches window ✓ GameWindow.java — JFrame, packs to panel size ✓ GamePanel.java — JPanel + Runnable game loop ✓ assets/sprites/ — PNG sprites pre-loaded ✓ assets/sounds/ — audio folder scaffolded	Engine Foundations ✓ Game loop via Thread.sleep(16) → ~60 FPS ✓ paintComponent() override for custom rendering ✓ 600 × 600 black game canvas ✓ Yellow Pac-Man circle placeholder rendered ✓ Window centered via setLocationRelativeTo(null)
--	--

KEY TECHNIQUES

Game Loop with Thread

GamePanel implements Runnable. The run() method loops infinitely, calling repaint() then sleeping 16ms — giving approximately 60 frames per second. This is the heartbeat of the entire game engine.

paintComponent() Override

All rendering happens inside paintComponent(Graphics g). Calling super.paintComponent(g) first clears the canvas to the background color, then custom shapes are drawn on top each frame.

JFrame + JPanel Architecture

GameWindow extends JFrame and simply adds a GamePanel. Calling pack() sizes the window to fit the panel's preferred size (600×600), and setLocationRelativeTo(null) centers it on the screen.

SOURCE CODE

App.java

java

```
public class App {
    public static void main(String[] args) {
        GameWindow window = new GameWindow();
        window.setVisible(true);
    }
}
```

GameWindow.java

java

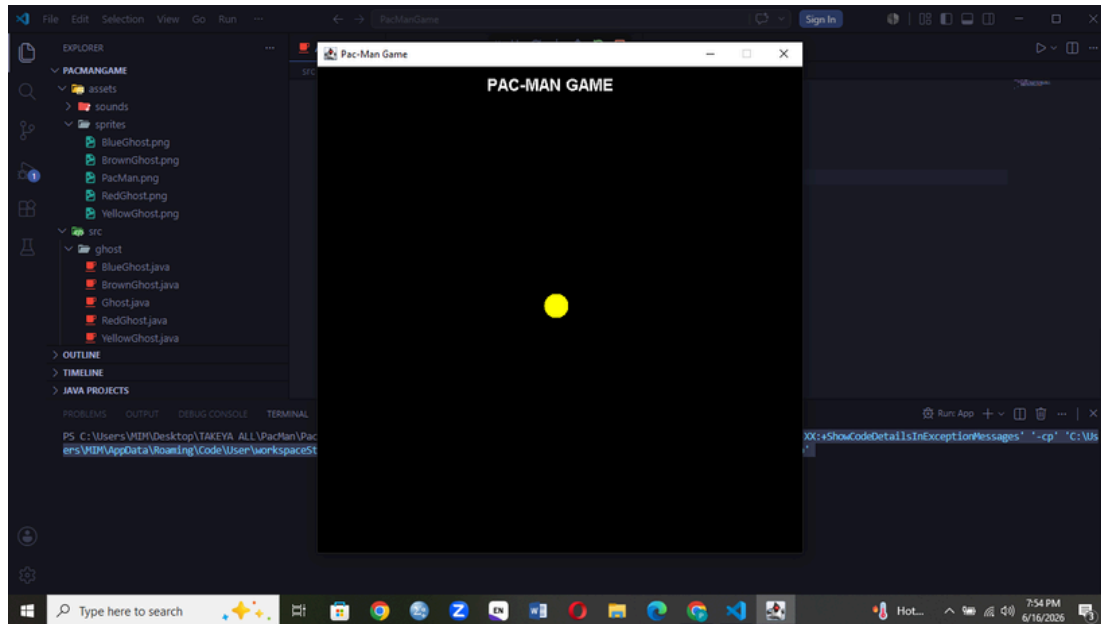
```
import javax.swing.JFrame;
public class GameWindow extends JFrame {
    public GameWindow() {
        setTitle("Pac-ManGame");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);
        GamePanel panel = new GamePanel();
        add(panel);
        pack();
        setLocationRelativeTo(null);
    }
}
```

GamePanel.java

java

```
import javax.swing.*;
import java.awt.*;
public class GamePanel extends JPanel implements Runnable {
    private Thread gameThread;
    public GamePanel() {
        setBackground(Color.BLACK);
        setPreferredSize(new Dimension(600, 600));
        setFocusable(true);
        gameThread = new Thread(this);
        gameThread.start();
    }
    @Override
    public void run() {
        while (true) {
            repaint();
            try { Thread.sleep(16); }
            catch (InterruptedException e) { e.printStackTrace(); }
        }
    }
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.YELLOW);
        g.fillOval(280, 280, 30, 30);
        g.setColor(Color.WHITE);
        g.setFont(new Font("Arial", Font.BOLD, 20));
        g.drawString("PAC-MAN GAME", 210, 30);
    }
}
```

OUTPUT



10-WEEK PROGRESS

Week 1 — Setup & Window

✓

Week 2 — Maze Design

100%

Weeks 3–4 — Pac-Man + Scoring

0%

Weeks 5–7 — Ghosts + AI

0%

Weeks 8–10 — Polish + Ship

0%

0%

UP NEXT

W2

Maze Design

Build the 2D grid that defines the entire game world — walls, paths, dots, and power pellets.

Maze.java · `int[][] grid` · Tile: 24px · 21×19 map · Blue walls · Dot circles